

Kohler AI Bathroom Designer

Prompts, System Instructions & Workflow Documentation

1. Overview: How AI Is Used In This System

The application is built as three deliberately separate layers so that every product recommendation stays explainable and grounded in the real Kohler catalog, rather than being freely generated by an LLM. AI is used only where it adds genuine value: understanding free-text user input, and turning structured results back into natural language.

Layer	Role	AI Involved?
RAG Retrieval	Embeds the full product catalog once at startup; ranks products by semantic similarity to the customer's described style.	Yes -> embedding model
Recommendation Engine	Filters by budget, applies the small-room bathtub-exclusion rule, and picks the final price-tier bundle.	No - >deterministic code
Conversational Layer	Extracts structured constraints from free text; narrates the final bundle in natural language.	Yes -> chat LLM

This separation is the core design principle: the LLM is never asked to invent a product name, a price, or a dimension. It only interprets language and explains results the deterministic engine already computed from the real catalog data.

2. RAG Retrieval Workflow

Embedding model: NVIDIA NIM hosted embedding model (configurable via NVIDIA_EMBED_MODEL), called through the OpenAI-compatible /v1/embeddings endpoint.

Indexing (runs once at server startup): every product across all 7 catalog files (toilets, tanks, faucets, showers, bathtubs, vanities, mirrors) is converted into a single descriptive string and embedded as a “passage”. The exact template used per product is:

```
describeProduct(p) = `${collection} ${name} ${type}
${category} ${description},    finish: ${finish}, price
tier: ${tier}`
```

Query time: the customer's own style description (captured verbatim by the extraction prompt below, e.g. “spa-like and calming with natural stone textures”) is embedded once as a “query” and compared via cosine similarity against every product in a category. For each category, only the top 60% most semantically relevant products (minimum 2) are kept as eligible candidates before the deterministic engine picks a specific price tier from that relevant pool.

This is what lets the system correctly surface muted/natural-finish products for a “spa-like” request even when the text never says “zen” or “natural” literally - true semantic matching rather than keyword substring matching.

3. System Prompt: Constraint Extraction

Function: extractSlots(messages) | **Called:** on every user message, reading the full conversation so far | **Output:** a JSON object merged into session state (only non-null fields overwrite existing known values).

Full system prompt:

You extract structured constraints from a conversation about designing a bathroom. Return ONLY a JSON object, no prose, no markdown fences, with exactly these keys:

```
{
  "widthFt": number or null,
  "lengthFt": number or null,
  "budget": number or null,  "theme": one of
"minimalist" | "luxury" | "zen" | null,
  "themeDescription": string or null,
  "showerWidthFt": number or null,
  "showerLengthFt": number or null,
  "wantsBathtub": true | false | null
}
```

Rules:

- widthFt/lengthFt: only set if the user gave bathroom dimensions in feet. - If given in a different unit, convert to feet. -
- budget: total budget in Indian Rupees as a plain number (e.g. "3 lakh" -> 300000, "50k" -> 50000).
- theme: map "minimalist", "modern", "simple" -> "minimalist"; "luxury", "classic", "premium", "opulent" -> "luxury"; "zen", "spa", "calm", "japanese", "natural" -> "zen".
- If the user hasn't expressed a style preference, use null. -
- themeDescription: the user's own words describing the look/feel/style they want (lightly cleaned up). Separate from the normalized theme field above and preserves nuance the enum loses. Null if not described. -
- showerWidthFt/showerLengthFt: the size of the shower area specifically (not the whole bathroom), only in feet. A single number is treated as a square. Only set when clearly answering about the shower, not the room. -
- wantsBathtub: only set true/false if the user has clearly answered a question about wanting a bathtub. Otherwise null. - Never guess a value that was not stated or clearly implied.

If you need to reason about the input, keep it brief. Then, as the very last thing in your response, output the JSON object on its own line with no other text after it. [3 worked examples included in the actual prompt - see repo source]

The JSON object must be the last thing you output. No text after it.

Model configuration: temperature 0, max_tokens 1000, chat_template_kwargs.enable_thinking=false (suppresses chain-of-thought on reasoning-capable NVIDIA models so the full token budget goes to the answer, not visible reasoning). When I Built the first prototype with the thinking process on, All my tokens were exhausted in the thinking process itself and very ambiguous results were produced

Robustness note: some NVIDIA-hosted models emit reasoning text even when instructed not to. The parser therefore extracts the first {...} block found anywhere in the response, rather than requiring the entire output to be pure JSON, and falls back to an empty object (leaving prior known slots untouched) if no valid JSON can be found.

4. System Prompt: Bundle Narration

Function: narrateBundle({bundle, slots, isSmallRoom, overBudget})

Called: once, after the deterministic engine has produced the final bundle.

Output: 3-5 sentences of natural-language explanation shown to the customer.

System prompt template (bracketed parts are filled in per-conversation):

You are a warm, concise bathroom design consultant. Explain the recommended product bundle in 3-5 sentences of natural prose. Reference specific products by name. Mention the total price. [If room is small: "Mention that a bathtub was left out because the room is compact."] [If customer declined a tub: "Mention the bathtub was skipped since the customer chose not to include one."] [If bundle exceeds budget: "Gently mention this bundle slightly exceeds the stated budget and is the closest available fit."] Do not invent any product names, prices, or specs beyond what's given below. Keep it grounded and specific, not generic marketing copy.

User message passed alongside it (constructed from the engine's actual output, never from model imagination): room size, budget, theme (plus verbatim style description if given), a line per bundle role with product name, finish, and real price in Rupees, and the computed total.

Model configuration: temperature 0.6 (some natural variation is fine here, unlike extraction), max_tokens 800, same thinking-suppression flag as above.

5. Deterministic Rules (Not AI - Included For Completeness)

These decisions are plain code, not model output, and are documented here to make clear which parts of the system are and are not AI-driven:

- Rooms under 35 sq ft never include a bathtub - excluded automatically, no question asked.
- Rooms 35 sq ft or larger are explicitly asked whether to include a bathtub; the engine only includes one if the answer was yes.
- Of each category, the top 60% of products by RAG relevance score (minimum 2) are eligible; within that pool, three price-sorted tiers are built and the highest-total tier that still fits the stated budget is selected.
- The LLM is never in the loop for price arithmetic, budget comparison, or product selection - only for language understanding (extraction) and language generation (narration).

6. Development Workflow Summary

The solution was built iteratively through a conversational AI-assisted development process:

(1) source catalog data was extracted and cross-referenced from Kohler's official India price book and retail spec-sheet PDFs into structured, price-and-dimension-verified JSON files per category;

(2) a deterministic recommendation engine was built and unit-tested against the catalog;

(3) RAG retrieval was layered in to replace keyword-based theme matching with real semantic search;

(4) the NVIDIA LLM layer was added for constraint extraction and narration, with prompts iteratively hardened against real failure modes (JSON truncation on reasoning models, deprecated model/endpoint names) encountered during development;

(5) the React/Vite frontend was built as a split-screen chat + live interactive 2D floor plan, with drag, rotate, alternate layout templates, and delete-with-live-bill-update added incrementally.

7. The AI Tools Used

User Interface and Design – <https://lovable.dev>

General Workflow/Plan for the project - <https://chatgpt.com>

The main development and code – <https://claude.com>

8. Resources Used for the Products displayed:

The Price Book:- [Price Book 2026](#)

The Retail Book(For the dimensions of the products):- [Retail Book](#)

Note:- I have referred to these catalogs and retrieved only few products from different categories, price ranges and combined each product's info into a JSON format for the LLM to perform RAG on the JSON files, I have not used the complete catalog